

Table of Contents

AI Use Declaration	3
1 Project Purpose	4
1.1 Context & Scope boundaries	4
1.2 Tech Stack	4
1.3 Architecture Overview	4
2 Functional Requirements	5
2.1 Blog Posts	5
2.2 Projects	5
2.3 Freelance work	5
2.4 Authentication	5
3 Non-Functional Requirements	6
3.1 Security	6
3.2 Performance	6
3.3 Reliability	6
3.4 Maintainability	6
4 API Specification	7
4.1 Authentication — Public	7
4.2 Blog Posts — Public	7
4.3 Blog Posts — Admin	8
4.4 Projects — Public	8
4.5 Projects — Admin	9
4.6 Freelance Work — Public	9
4.7 Freelance Work — Admin	10
5 Database Schema Design	11
5.1 Flyway Migration Scripts Structure	11
5.2 DDL Schema Definition	11
5.2.1 V1__create_users_table.sql	11
5.2.2 V2__create_blog_posts_table.sql	11
5.2.3 V3__create_projects_table.sql	12
5.2.4 V4__create_freelance_table.sql	12
6 Development Plan	13
6.1 Sprint 1: System Foundation & Core Database (Database & Structure)	13
6.2 Sprint 2: Resource Engines & Public Access (Public Features)	13
6.3 Sprint 3: AAA Security Layer & Protected Management (Admin Control)	13
6.4 Sprint 4: Hardening, DevOps, & Production Infrastructure	13

Abstract

This document details the architectural blueprint for a REST API back-end supporting a personal portfolio website. Built using Java Spring Boot and PostgreSQL, the API serves dynamic content to a decoupled Next.js front-end hosted on Netlify providing public read access to portfolio items while securing admin management features.

AI Use Declaration

To remain integral, Generative Artificial Intelligence (Gemini) was utilised to assist in drafting this Project Document. AI assistance was strictly used for:

- Structuring and formatting technical constraints, functional requirements, and sprint milestones into clean compiled $\text{\LaTeX}$.
- Optimising the PostgreSQL DDL schema syntax and relational mapping design.
- Refining descriptions within the technical roadmap.

The architectural decision, tech stack selection, API endpoints definitions, and database relationships were created and designed by the Author (Eduard Teodor).

1 Project Purpose

The primary purpose of this system is to provide a centralised, self-hosted Content Management System. By decoupling the back-end from the front-end presentation layer, the architecture ensures high availability, static generation capabilities on the client side via Next.js, and robust security for data modification.

1.1 Context & Scope boundaries

The scope of this initial development phase includes user authentication, blog post publishing workflows (including draft and starred sorting behaviors), project showcase management with featured priorities, and historical freelance engagement tracking. External integrations are strictly limited to the Netlify front-end origin.

1.2 Tech Stack

- **Java & Spring** – Java 25.0.3, Spring Boot 4.0.6
- **PostgreSQL** – PostgreSQL 16.0
- **Gradle** – Kotlin DSL, Build Tool
- **Docker** – Containerization

1.3 Architecture Overview

The system will follow a three-tier architecture with a feature-based package structure:

Next.js (Netlify) → Spring Boot API → PostgreSQL

2 Functional Requirements

The following requirements define what the system must do:

2.1 Blog Posts

- FR-1:** The system shall allow an admin to create blog posts
- FR-2:** The system shall allow an admin to update an existing blog posts
- FR-3:** The system shall allow an admin to delete blog posts
- FR-4:** The system shall allow an admin to view all posts including drafts
- FR-5:** The system shall allow an admin to star a blog
- FR-6:** The system shall display only published posts to public users
- FR-7:** The system shall allow public users to retrieve a post by its URL slug
- FR-8:** The system shall return the 3 most recently published posts for the homepage
- FR-9:** The system shall return starred blogs in front of 3 most recently published posts for the homepage

2.2 Projects

- FR-10:** The system shall allow an admin to create, update, and delete projects
- FR-11:** The system shall display only published projects to public users
- FR-12:** The system shall allow filtering projects by featured status
- FR-13:** The system shall return the top 3 featured projects for the homepage
- FR-14:** The system shall allow retrieving a single project by UUID

2.3 Freelance work

- FR-15:** The system shall allow an admin to create, update, and delete freelance work entries
- FR-16:** The system shall display only published freelance entries to public users
- FR-17:** The system shall support optional client testimonials per entry

2.4 Authentication

- FR-18:** The system shall provide a login endpoint that returns a JWT token
- FR-19:** The system shall validate JWT tokens on all admin requests
- FR-20:** The System shall reject unauthenticated requests to admin routes with 401

3 Non-Functional Requirements

3.1 Security

- NFR-1:** All admin routes must require a valid JWT token
- NFR-2:** JWT tokens must expire after a defined period
- NFR-3:** Passwords must be stored as bcrypt hashes, never plaintext
- NFR-4:** CORS must only permit requests from the Netlify frontend origin
- NFR-5:** CSRF protection disabled (stateless JWT architecture)

3.2 Performance

- NFR-6:** Public API endpoints must respond within 200ms under normal load
- NFR-7:** Database queries must use indexes on frequently queried columns

3.3 Reliability

- NFR-8:** Database schema changes must be managed exclusively through Flyway
- NFR-9:** The application must log errors with sufficient detail for debugging
- NFR-10:** The API must return meaningful HTTP status codes on all responses

3.4 Maintainability

- NFR-11:** Code must follow feature-based package structure
- NFR-12:** Each feature must have its own controller, service, and repository layer
- NFR-13:** No business logic in controllers; controllers delegate to services only

4 API Specification

This section outlines the preliminary endpoints for the system's REST API. The route matrix is made of domain feature boundaries and explicitly states the authentication mechanisms required for resource access. Public endpoints require no authorisation token, while administrative endpoints and data access routes require a valid JSON Web Token.

4.1 Authentication — Public

Method	Route	Auth	Description	Response
POST	/api/auth/login	None	Authenticates admin credentials and returns a signed JWT access token	200 TokenResponse / 401 Unauthorized

4.2 Blog Posts — Public

Method	Route	Auth	Description	Response
GET	/api/blog	None	Retrieves all published blog posts	200 List<BlogPost>
GET	/api/blog/homepage	None	Returns the top 3 homepage posts (starred posts sorted first, followed by the most recently published)	200 List<BlogPost>
GET	/api/blog/{slug}	None	Retrieves a single published blog post by its unique URL slug	200 BlogPost / 404 Not Found

4.3 Blog Posts — Admin

Method	Route	Auth	Description	Response
GET	/api/admin/blog	JWT	Retrieves all blog posts from the database, including unpublished drafts	200 List<BlogPost> / 401 Unauthorized
POST	/api/admin/blog	JWT	Creates a new blog post entry (can be saved as draft or published)	201 BlogPost / 400 Bad Request / 401 Unauthorized
PUT	/api/admin/blog/{uuid}	JWT	Updates the title, content, slug, or publication status of an existing post	200 BlogPost / 404 Not Found / 401 Unauthorized
DELETE	/api/admin/blog/{uuid}	JWT	Permanently removes a blog post entry from the database	204 No Content / 404 Not Found / 401 Unauthorized
PATCH	/api/admin/blog/{uuid}/star	JWT	Toggles the structural starred flag status of the specified blog post	200 BlogPost / 404 Not Found / 401 Unauthorized

4.4 Projects — Public

Method	Route	Auth	Description	Response
GET	/api/projects	None	Retrieves all published projects (supports optional filter parameter ?featured=true)	200 List<Project>
GET	/api/projects/homepage	None	Retrieves exactly the top 3 featured portfolio projects for the landing page showcase	200 List<Project>
GET	/api/projects/{uuid}	None	Retrieves a single published portfolio project using its unique UUID identifier	200 Project / 404 Not Found

4.5 Projects — Admin

Method	Route	Auth	Description	Response
POST	/api/admin/projects	JWT	Creates a new portfolio showcase project entry	201 Project / 400 Bad Request / 401 Unauthorized
PUT	/api/admin/projects/{uuid}	JWT	Modifies text field attributes, stack data, or status flags of an existing project	200 Project / 404 Not Found / 401 Unauthorized
DELETE	/api/admin/projects/{uuid}	JWT	Erases an existing portfolio showcase project entry from the persistent database	204 No Content / 404 Not Found / 401 Unauthorized

4.6 Freelance Work — Public

Method	Route	Auth	Description	Response
GET	/api/freelance	None	Retrieves all published freelance history records, embedding client testimonials where available	200 List<FreelanceEntry>

4.7 Freelance Work — Admin

Method	Route	Auth	Description	Response
POST	/api/admin/freelance	JWT	Logs a new freelance project contract block with optional client testimonial metrics	201 FreelanceEntry / 400 Bad Request / 401 Unauthorized
PUT	/api/admin/freelance/{uuid}	JWT	Updates descriptive entries, client details, or attached testimonial references	200 FreelanceEntry / 404 Not Found / 401 Unauthorized
DELETE	/api/admin/freelance/{uuid}	JWT	Purges a specific freelance transaction log out of the persistent repository	204 No Content / 404 Not Found / 401 Unauthorized

5 Database Schema Design

All schema changes are managed exclusively through Flyway versioned migrations. To maintain data integrity and accountability, every piece of content created in the application is tied back directly to the sole authenticating admin user created in the master user entity.

5.1 Flyway Migration Scripts Structure

Migrations are structured sequentially using Flyway naming standards within the workspace repository directory `src/main/resources/db/migration/`:

- **V1__create_users_table.sql** – Initializes the security credentials layout and seeds the initial sole admin user account row.
- **V2__create_blog_posts_table.sql** – Establishes core data modeling for published content and draft logs, bound to a user ID creator.
- **V3__create_projects_table.sql** – Establishes structures for portfolio showcases, tracking featured priorities and metadata flags.
- **V4__create_freelance_table.sql** – Handles freelance engagement archiving along with attached optional client text feedback metrics.

5.2 DDL Schema Definition

The following PostgreSQL-compliant scripts represent the preliminary database design and baseline production architecture:

5.2.1 V1__create_users_table.sql

```
1 CREATE TABLE IF NOT EXISTS users (  
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   username VARCHAR(100) NOT NULL UNIQUE,  
4   password_hash VARCHAR(255) NOT NULL,  
5   created_at TIMESTAMP DEFAULT NOW()  
6 );
```

5.2.2 V2__create_blog_posts_table.sql

```
1 CREATE TABLE IF NOT EXISTS blog_posts (  
2   id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
3   user_id UUID NOT NULL REFERENCES users(id),  
4   slug VARCHAR(255) NOT NULL UNIQUE,  
5   title VARCHAR(255) NOT NULL,  
6   content TEXT NOT NULL,  
7   excerpt VARCHAR(500),  
8   starred BOOLEAN DEFAULT false,  
9   published BOOLEAN DEFAULT false,  
10  published_at TIMESTAMP,  
11  created_at TIMESTAMP DEFAULT NOW(),  
12  updated_at TIMESTAMP DEFAULT NOW()  
13 );  
14  
15 CREATE INDEX idx_blog_posts_slug ON blog_posts(slug);  
16 CREATE INDEX idx_blog_posts_published ON blog_posts(published);  
17 CREATE INDEX idx_blog_posts_starred ON blog_posts(starred);
```

```
18      CREATE INDEX idx_blog_posts_published_at ON blog_posts(published_at
      DESC);
```

5.2.3 V3__create_projects_table.sql

```
1      CREATE TABLE IF NOT EXISTS projects (
2          id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3          user_id UUID NOT NULL REFERENCES users(id),
4          title VARCHAR(255) NOT NULL,
5          description TEXT NOT NULL,
6          summary VARCHAR(500),
7          tech_stack TEXT,
8          github_url VARCHAR(500),
9          live_url VARCHAR(500),
10         status VARCHAR(50),
11         featured BOOLEAN DEFAULT false,
12         published BOOLEAN DEFAULT false,
13         created_at TIMESTAMP DEFAULT NOW(),
14         updated_at TIMESTAMP DEFAULT NOW()
15     );
16
17     CREATE INDEX idx_projects_published ON projects(published);
18     CREATE INDEX idx_projects_featured ON projects(featured);
```

5.2.4 V4__create_freelance_table.sql

```
1      CREATE TABLE IF NOT EXISTS freelance_work (
2          id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
3          user_id UUID NOT NULL REFERENCES users(id),
4          client_name VARCHAR(255),
5          project_title VARCHAR(255) NOT NULL,
6          description TEXT NOT NULL,
7          services TEXT,
8          testimonial TEXT,
9          website_url VARCHAR(500),
10         featured BOOLEAN DEFAULT false,
11         published BOOLEAN DEFAULT false,
12         completed_at TIMESTAMP,
13         created_at TIMESTAMP DEFAULT NOW(),
14         updated_at TIMESTAMP DEFAULT NOW()
15     );
16
17     CREATE INDEX idx_freelance_published ON freelance_work(published);
18     CREATE INDEX idx_freelance_featured ON freelance_work(featured);
```

6 Development Plan

This development roadmap outlines the iterative delivery sequence of the portfolio API. Each sprint maps explicitly to specific Functional Requirements (**FR**) and Non-Functional Requirements (**NFR**) to ensure full architecture traceability.

6.1 Sprint 1: System Foundation & Core Database (Database & Structure)

- **Status** – Completed
- **Deliverables** – Project scaffolding setup with Java 25 and Spring Boot 4.0.6. Initialisation of local Docker runtime containers. Configuration of versioned Flyway schema scripts (V1 through V4) to build tables with proper tracking indices.
- **Traceability** – Covers architectural design choices, **NFR-6** (Database indexing), **NFR-8** (Flyway execution engine), and **NFR-11** / **NFR-12** (Feature-based package layering).

6.2 Sprint 2: Resource Engines & Public Access (Public Features)

- **Status** – In Progress
- **Deliverables** – Development of read-only business logic engines, repository querying interfaces, and controller endpoints. This includes specialized homepage sorting algorithms to prioritize starred blogs and handle project/freelance data mapping payloads.
- **Traceability** – Covers **FR-6** to **FR-9** (Public blog features), **FR-11** to **FR-14** (Public projects), **FR-16** (Public freelance listings), **NFR-7** (Under 200ms API response benchmark), and **NFR-13** (No business logic inside controllers).

6.3 Sprint 3: AAA Security Layer & Protected Management (Admin Control)

- **Status** – Planned
- **Deliverables** – Integration of the Spring Security filter chain. Setup of custom filters to validate stateless incoming user authentication payloads. Implement administrative endpoints allowing full secure modification operations (Create, Update, Delete, Patch) across resources.
- **Traceability** – Covers **FR-1** to **FR-5** (Admin blog control), **FR-10** (Admin project modifications), **FR-15** (Admin freelance control), **FR-17** to **FR-19** (JWT core actions), **NFR-1** / **NFR-2** (Stateless JWT token handling), **NFR-3** (BCrypt password salting), and **NFR-5** (CSRF bypass safety).

6.4 Sprint 4: Hardening, DevOps, & Production Infrastructure

- **Status** – Planned
- **Deliverables** – Securing network access controls by applying CORS restrictions to isolate outside consumption. Implementing global exception interceptors to enforce error output standards, finalizing containerized build variants, configuring application variables, and setting up persistent debugging logs.
- **Traceability** – Covers **NFR-4** (Cross-Origin restricted Netlify whitelisting), **NFR-9** (Verbose logging for system exceptions), and **NFR-10** (Standard HTTP Response payload mapping).